

Practice Problems of Week 8

Java Solutions • Inheritance • Method Overriding • Runtime Polymorphism

Key Highlights

Inheritance; method overriding; runtime polymorphism; common base class and specialized behavior; avoiding if-else type checking; polymorphic processing; and different rules for different derived classes.

Problem 1: The Canteen Billing Counter

Student discount: 10%. Staff discount: 5%. Guest: full bill plus ■10.

Java Solution

```
import java.util.*;

abstract class Customer {
    protected double amount;
    Customer(double amount) { this.amount = amount; }
    abstract double finalAmount();
    abstract String type();
}

class Student extends Customer {
    Student(double amount) { super(amount); }
    double finalAmount() { return amount * 0.90; }
    String type() { return "STUDENT"; }
}

class Staff extends Customer {
    Staff(double amount) { super(amount); }
    double finalAmount() { return amount * 0.95; }
    String type() { return "STAFF"; }
}

class Guest extends Customer {
    Guest(double amount) { super(amount); }
    double finalAmount() { return amount + 10; }
    String type() { return "GUEST"; }
}

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = Integer.parseInt(sc.nextLine().trim());
        double total = 0;
        for (int i = 0; i < n; i++) {
            String[] d = sc.nextLine().trim().split("\\s+");
            double amount = Double.parseDouble(d[1]);
            Customer c;
            switch (d[0]) {
                case "STUDENT": c = new Student(amount); break;
                case "STAFF": c = new Staff(amount); break;
                default: c = new Guest(amount);
            }
            System.out.printf("%s: %.2f%n", c.type(), c.finalAmount());
            total += c.finalAmount();
        }
        System.out.printf("Total: %.2f%n", total);
    }
}
```

Problem 2: The Campus Parking Charge Calculator

Bike: ■10 per hour. Car: ■30 for the first hour plus ■20 for each additional hour. Truck: ■50 per hour, minimum ■100.

Java Solution

```
import java.util.*;

abstract class Vehicle {
    protected int hours;
    Vehicle(int hours) { this.hours = hours; }
    abstract double charge();
    abstract String type();
}

class Bike extends Vehicle {
    Bike(int h) { super(h); }
    double charge() { return hours * 10.0; }
    String type() { return "BIKE"; }
}

class Car extends Vehicle {
    Car(int h) { super(h); }
    double charge() { return 30 + (hours - 1) * 20.0; }
    String type() { return "CAR"; }
}

class Truck extends Vehicle {
    Truck(int h) { super(h); }
    double charge() { return Math.max(100, hours * 50.0); }
    String type() { return "TRUCK"; }
}

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = Integer.parseInt(sc.nextLine().trim());
        double total = 0;
        for (int i = 0; i < n; i++) {
            String[] d = sc.nextLine().trim().split("\\s+");
            int hours = Integer.parseInt(d[1]);
            Vehicle v;
            switch (d[0]) {
                case "BIKE": v = new Bike(hours); break;
                case "CAR": v = new Car(hours); break;
                default: v = new Truck(hours);
            }
            System.out.printf("%s: %.2f%n", v.type(), v.charge());
            total += v.charge();
        }
        System.out.printf("Total: %.2f%n", total);
    }
}
```

Problem 3: The Hostel Electricity Bill

Single: ■8 per unit. Shared: ■6 per unit divided by occupants. AC: ■10 per unit plus ■200.

Java Solution

```
import java.util.*;

abstract class Room {
    protected int units;
    Room(int units) { this.units = units; }
    abstract double bill();
    abstract String type();
}

class SingleRoom extends Room {
    SingleRoom(int u) { super(u); }
    double bill() { return units * 8.0; }
    String type() { return "SINGLE"; }
}

class SharedRoom extends Room {
    private int occupants;
    SharedRoom(int u, int o) { super(u); occupants = o; }
    double bill() { return units * 6.0 / occupants; }
    String type() { return "SHARED"; }
}

class AcRoom extends Room {
    AcRoom(int u) { super(u); }
    double bill() { return units * 10.0 + 200; }
    String type() { return "AC"; }
}

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = Integer.parseInt(sc.nextLine().trim());
        double total = 0;
        for (int i = 0; i < n; i++) {
            String[] d = sc.nextLine().trim().split("\\s+");
            int units = Integer.parseInt(d[1]);
            Room r;
            if (d[0].equals("SINGLE")) r = new SingleRoom(units);
            else if (d[0].equals("SHARED"))
                r = new SharedRoom(units, Integer.parseInt(d[2]));
            else r = new AcRoom(units);
            System.out.printf("%s: %.2f%n", r.type(), r.bill());
            total += r.bill();
        }
        System.out.printf("Total: %.2f%n", total);
    }
}
```

Problem 4: The Festival Bonus Calculator

Full-time: 10% of salary. Part-time: 5% of salary. Intern: fixed bonus of ■2,000.

Java Solution

```
import java.util.*;

abstract class Employee {
    protected String name;
    protected double salary;
    Employee(String n, double s) { name = n; salary = s; }
    abstract double bonus();
}

class FullTime extends Employee {
    FullTime(String n, double s) { super(n, s); }
    double bonus() { return salary * 0.10; }
}

class PartTime extends Employee {
    PartTime(String n, double s) { super(n, s); }
    double bonus() { return salary * 0.05; }
}

class Intern extends Employee {
    Intern(String n, double s) { super(n, s); }
    double bonus() { return 2000; }
}

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = Integer.parseInt(sc.nextLine().trim());
        double total = 0;
        for (int i = 0; i < n; i++) {
            String[] d = sc.nextLine().trim().split("\\s+");
            double salary = Double.parseDouble(d[2]);
            Employee e;
            switch (d[0]) {
                case "FULLTIME": e = new FullTime(d[1], salary); break;
                case "PARTTIME": e = new PartTime(d[1], salary); break;
                default: e = new Intern(d[1], salary);
            }
            double b = e.bonus();
            System.out.printf("%s: %.2f%n", d[1], b);
            total += b;
        }
        System.out.printf("Total Bonus: %.2f%n", total);
    }
}
```

Problem 5: The Streaming Plan Renewal Reminder

Basic: 30 days. Standard: 90 days. Premium: 365 days. `LocalDate.plusDays()` handles leap years and year changes.

Java Solution

```
import java.time.LocalDate;
import java.util.*;

abstract class Plan {
    protected String name;
    protected LocalDate startDate;
    Plan(String n, LocalDate d) { name = n; startDate = d; }
    abstract int validityDays();
    LocalDate renewalDate() { return startDate.plusDays(validityDays()); }
}

class BasicPlan extends Plan {
    BasicPlan(String n, LocalDate d) { super(n, d); }
    int validityDays() { return 30; }
}

class StandardPlan extends Plan {
    StandardPlan(String n, LocalDate d) { super(n, d); }
    int validityDays() { return 90; }
}

class PremiumPlan extends Plan {
    PremiumPlan(String n, LocalDate d) { super(n, d); }
    int validityDays() { return 365; }
}

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = Integer.parseInt(sc.nextLine().trim());
        for (int i = 0; i < n; i++) {
            String[] d = sc.nextLine().trim().split("\\s+");
            LocalDate start = LocalDate.parse(d[2]);
            Plan p;
            switch (d[0]) {
                case "BASIC": p = new BasicPlan(d[1], start); break;
                case "STANDARD": p = new StandardPlan(d[1], start); break;
                default: p = new PremiumPlan(d[1], start);
            }
            System.out.println(p.name + ": " + p.renewalDate());
        }
    }
}
```